

API 规范文档

1. 设计依据

本 API 规范基于以下内容统一整理：

- P1 功能需求中的 PO 能力
- P2 架构设计文档中的接口草案
- 当前后端控制器命名：`AuthController`、`TaskController`、`TaskCommentController`、`TaskReviewController`、`MessageController`、`UserController`、`FeedbackController`、`AdminController`

2. 统一约定

2.1 Base URL

`/api`

2.2 认证方式

- 登录后使用 `Authorization: Bearer <token>`
- 管理端接口要求用户角色为 `ADMIN`

2.3 统一响应结构

```
{
  "code": 0,
  "message": "ok",
  "data": {}
}
```

2.4 统一错误结构

```
{
  "code": 40001,
  "message": "任务已被接单",
  "data": null
}
```

3. 错误码定义

错误码	含义
0	成功

错误码	含义
40000	请求参数错误
40001	业务状态冲突
40100	未登录或令牌失效
40300	无权限访问
40400	资源不存在
40900	唯一约束冲突
42200	内容审核未通过
50000	服务器内部错误

4. 核心接口

4.1 用户注册

POST /api/auth/register

请求参数：

字段	类型	必填	说明
studentId	string	是	学号
name	string	是	真实姓名
email	string	是	校园邮箱
password	string	是	明文输入，服务端加密存储

请求示例：

```
{
  "studentId": "20230001",
  "name": "张三",
  "email": "20230001@campus.edu.cn",
  "password": "12345678"
}
```

成功响应：

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "token": "jwt-token",
    "userProfile": {
```

```
    "id": 1,
    "studentIdMasked": "2023****",
    "name": "张三",
    "role": "USER",
    "status": "ACTIVE"
  }
}
```

失败场景：

- 学号已注册：40900
- 邮箱已注册：40900
- 参数不合法：40000

4.2 用户登录

POST /api/auth/login

请求参数：

字段	类型	必填	说明
studentId	string	是	学号
password	string	是	密码

成功响应字段：

- token
- refreshToken
- userProfile

失败场景：

- 用户不存在：40400
- 密码错误：40100
- 用户被禁用：40300

4.3 发布需求

POST /api/tasks

请求参数：

字段	类型	必填	说明
title	string	是	标题
description	string	是	详细描述
category	string	是	如跑腿代办、二手闲置、恋爱交友

字段	类型	必填	说明
taskMode	string	是	task 或 topic
contactInfo	string	否	达到允许阶段后可展示
locationText	string	否	地点文本
timeText	string	否	时间说明
rewardTitle	string	否	奖励标题
rewardText	string	否	奖励说明
expiresAt	string	否	过期时间

成功响应：

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "id": 101,
    "title": "帮取快递",
    "category": "跑腿代办",
    "taskMode": "task",
    "status": "OPEN",
    "reviewStatus": "approved"
  }
}
```

失败场景：

- 未实名认证：40300
- 内容审核不通过：42200
- 参数缺失：40000

4.4 浏览需求列表

GET /api/tasks

查询参数：

字段	类型	必填	说明
type	string	否	兼容旧筛选字段
category	string	否	分类筛选
status	string	否	状态筛选
keyword	string	否	关键字

字段	类型	必填	说明
page	integer	否	页码，默认 1
size	integer	否	每页条数，默认 10

成功响应：

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "records": [
      {
        "id": 101,
        "title": "帮取快递",
        "category": "跑腿代办",
        "taskMode": "task",
        "status": "OPEN",
        "likeCount": 3
      }
    ],
    "page": 1,
    "size": 10,
    "total": 35
  }
}
```

失败场景：

- 分页参数非法：40000

4.5 接单

POST /api/tasks/{id}/accept

路径参数：

字段	类型	必填	说明
id	integer	是	任务 ID

成功响应：

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "id": 101,
    "status": "IN_PROGRESS",
    "participantCount": 1
  }
}
```

```
    }  
  }  
}
```

失败场景：

- 任务不存在：40400
- 已被接单：40001
- 自己不能接自己的单：40001
- 话题类型不允许接单：40001

4.6 查看订单详情

GET /api/tasks/{id}

路径参数：

字段	类型	必填	说明
id	integer	是	任务 ID

成功响应关键字段：

- 任务主体信息
- 发布者脱敏信息
- 当前状态
- 点赞数
- 评论数
- 当前参与者摘要

失败场景：

- 资源不存在：40400
- 审核未通过且当前用户无权限查看：40300

4.7 提交评价

POST /api/tasks/{id}/reviews

请求参数：

字段	类型	必填	说明
rating	integer	是	1-5 星
content	string	否	评价内容
revieweeId	integer	是	被评价人

成功响应：

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "id": 9001,
    "taskId": 101,
    "reviewerId": 1,
    "revieweeId": 2,
    "rating": 5
  }
}
```

失败场景：

- 任务尚未完成：40001
- 重复评价：40900
- 不是任务参与者：40300

4.8 评论任务/话题

POST /api/tasks/{id}/comments

请求参数：

字段	类型	必填	说明
content	string	是	评论内容
anonymous	boolean	否	是否匿名显示
parentId	integer	否	回复的父评论 ID

失败场景：

- 内容为空：40000
- 内容审核失败：42200

4.9 提交反馈/仲裁

POST /api/feedback

请求参数：

字段	类型	必填	说明
type	string	是	DISPUTE 或 SUGGESTION
title	string	是	标题
content	string	是	详细内容
taskId	integer	否	关联任务 ID

4.10 消息列表

GET /api/messages

查询参数：

字段	类型	必填	说明
page	integer	否	页码
size	integer	否	页大小
status	string	否	UNREAD / READ

5. AI 生成接口草案的审查记录

检查项	发现的问题	修正结果
接口命名不一致	AI 初稿混用了 /task、/tasks、/order	统一为 REST 风格复数资源：/tasks、/messages、/feedback
缺少错误处理	初稿多数只写成功响应	为每个核心接口补充统一错误码与失败场景
参数校验不完整	对 rating、page、taskMode 等缺少合法值范围	在规范中明确校验条件
安全问题	初稿未写 Bearer Token、角色控制、禁用用户限制	增加认证方式、角色边界与资源访问限制

6. 结论

当前 API 规范已覆盖 P3 要求的 6 个核心接口，并与现有项目控制器命名、P2 路径设计和数据库结构保持一致。